

Reinforcement Learning With Reward Machines in Stochastic Games

Reinforcement Learning Course - Final Project — 06-2026

Matteo Liotta

`matteo.liotta@studenti.units.it`

Mattia Simonutti

`mattia.simonutti@studenti.units.it`

University of Trieste

Department of Mathematics and Geosciences

Problem Setting

- **Goal:** Learn to the optimal best-response strategy at a **Nash equilibrium** for each agent in a **general-sum game setting**.
- **Problem:** If multi-agent complex stochastic games induce system **non-Markovianity**, how to handle **non-Markovian** reward functions?
- **Proposed Approach:** Usage of **Reward Machines** to incorporate the high-level game knowledge and **converting non-Markovian reward functions** into **standard Markovian ones** [1].

Basic Definitions

Stochastic Games with Non-Markovian Rewards

Definition — Two-Player General-Sum Stochastic Game (SG)

Modeled as the tuple $\mathcal{G} = (S, s_I, A_e, A_a, p, R_e, R_a, \gamma)$, where:

- **Finite State Space (S):** consisting of **states** $s = [s_e; s_a]$, with s_e, s_a substates for *Ego* and *Adv* agents respectively. $s_I \in S$ is the **initial** state.
- **Action Spaces:** Each agent has its own finite set of actions A_e and A_a .
- **Probabilistic Transition Function:** $p : S \times A_e \times A_a \times S \rightarrow [0, 1]$, i.e. $p(s' \mid s, a_e, a_a)$
- **Non*-Markovian Reward functions:** $R_e \text{ or } a : (S \times A_e \times A_a)^+ \times S \rightarrow \mathbb{R}$ map from **complete trajectory histories** to a real value.

(*) Usual definition assumes system Markovianity.

Trajectories and Rewards

- **Trajectory** (at step k): sequence of **states** and **actions** defined as

$$t_{0:k} = s_0 a_{\epsilon,0} a_{\alpha,0} s_1 \dots s_{k-1} a_{\epsilon,k-1} a_{\alpha,k-1} s_k \quad \text{with} \quad s_0 = s_I$$

- **Reward Sequence:** Each agent $i \in \{\epsilon, \alpha\}$ receives a sequence $r_{i,1} \dots r_{i,k}$, where each value is determined by the **history up to that step**:

$$r_{i,j} = R_i(s_0 a_{\epsilon,0} a_{\alpha,0} \dots s_j a_{\epsilon,j} a_{\alpha,j} s_{j+1}) = R_i(t_{0:j+1}) \quad \text{for each } j \leq k$$

- **Shared Observation:** Both agents **observe the same** trajectory.
- **Discounted Cumulative Reward:** Given a trajectory $t_{0:k}$, agents achieves $\sum_{j=1}^k \gamma^{j-1} r_{i,j}$, where $\gamma \in (0, 1]$

Strategies and Objectives

- **Markovian Strategy:** Agents are equipped with a $\pi_i : S \times A_i \rightarrow [0, 1]$ $i \in \{\epsilon, \alpha\}$ strategy, with $\pi_i(a_i|s)$ probability of agent i selecting action a_i from state s .
- **Objective:** Maximize the **expected** discounted cumulative reward given policy of both agents, from both point of view.

Definition — SG Expected Discounted Cumulative Reward

Given agent $i \in \{\epsilon, \alpha\}$, the expected DCR under policies π_ϵ and π_α is defined as:

$$\tilde{v}_i(s, \pi_\epsilon, \pi_\alpha) = \sum_{t=0}^{\infty} \gamma^t \mathbb{E}(r_{i,t} | \pi_\epsilon, \pi_\alpha, s_0 = s)$$

Nash Equilibrium

Since in General-Sum Stochastic Games each agent has its own reward, there is no single policy that is globally optimal for everyone.

Therefore a Nash equilibrium gives a stability criterion:

Definition — Nash Equilibrium in Stochastic Games

For a stochastic game $\mathcal{G} = (S, s_I, A_e, A_a, R_e, R_a, \gamma, p)$, the strategies of the ego agent, π_e^* , and the adversarial agent, π_a^* , are at a **Nash equilibrium** if for any π_e, π_a :

$$\begin{aligned}\tilde{v}_e(s, \pi_e^*, \pi_a^*) &\geq \tilde{v}_e(s, \pi_e, \pi_a^*) \\ \tilde{v}_a(s, \pi_e^*, \pi_a^*) &\geq \tilde{v}_a(s, \pi_e^*, \pi_a)\end{aligned}$$

With $*$ defining a fixed point of stability

Definition — Reward Machine (RM)

Mealy machine ($\approx$ DFA) designed to **encode non-Markovian rewards** with **state transitions** and **output functions**.

$$\mathcal{A} = (V, v_I, 2^{\mathcal{P}}, \mathbb{R}, \delta, \sigma)$$

- **Set of RM states (V):** Finite collection of internal **game** evolution tracking **states**, with v_I as the **initial** state.
- **Input Alphabet ($2^{\mathcal{P}}$):** Set of possible combinations of **propositional variables** of $p \in \mathcal{P}$, which encodes high-level environmental knowledge/events.
- **Transition function: Deterministic** game state progression $\delta : V \times 2^{\mathcal{P}} \rightarrow V$.
- **Output function:** $\sigma : V \times 2^{\mathcal{P}} \rightarrow \mathbb{R}$, associates values (**rewards**) along active trajectories.

Connecting SG and RMs

A link between RM and SG is required

Labelling Function (external to RMs)

$$L : S \times A_e \times A_a \times S \rightarrow 2^{\mathcal{P}}$$

$l_t = L(s_t, a_{a,t}, a_{e,t}, s_{t+1})$ is the **set of proposition** in $\mathcal{P}$ having **true** evaluation at step t .

RM can encode non-Markovian RF of a stochastic game

Given a SG, the RM $\mathcal{A}_i$ **encodes** the non-markovian reward function R_i of the game, meaning that **for each trajectory** $s_0 a_{e,0} a_{a,0} s_1 \dots s_k a_{e,k} a_{a,k} s_{k+1}$ (i.e. $l_0 l_1 \dots, l_k$) it outputs the **reward sequence** $r_{i,0} r_{i,1} \dots, r_{i,k}$.

I.e. the RM maps **any sequence of events to a sequence of rewards**.

Visual Examples of RMs and L

Reference paper focuses on **two-agent ZS-SG** where each agent potentially given a different task to complete. where task completion **depends on the other's agent behavior**.

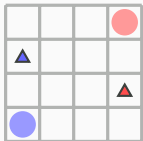


Figure. Example Grid World

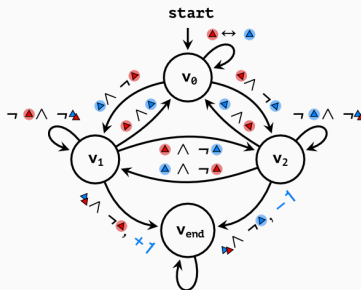


Figure. Example Associated RM

$$L(s_t, a_a, a_e, s_{t+1}) = \{\text{red triangle}, \text{blue triangle}\}$$

Equation. Example Labelling function output

Q-learning with RM for SG

Markovian formulation of SG with RMs

Definition — Stochastic Game with Reward Machines (SGRM)

Given an SG $\mathcal{G}$ and RMs $\mathcal{A}_e, \mathcal{A}_a$, a SGRM is defined as a product stochastic game:

$$\mathcal{H} = (S', s'_I, A_e, A_a, p', R'_e, R'_a, \gamma)$$

- **Augmented State Space:** $S' = S \times V_e \times V_a$. An **augmented state** is $s' = (s, v_e, v_a)$.
- **Transition Probabilities:** $p'(s', a_e, a_a, \bar{s}') = p(s, a_e, a_a, \bar{s})$ if $v'_i = \delta_i(v_i, L(s, a_e, a_a, \bar{s}))$ for both $i \in \{e, a\}$, and 0 otherwise.
- **Markovian Reward Functions:** $R'_i(s', a_e, a_a, \bar{s}') = \sigma_i(v_i, L(s, a_e, a_a, \bar{s}))$, $\forall i$

Why the SGRM formulation?

The newly introduced Stochastic Game $\mathcal{H}$ is proved to have a **Markovian rewards functions** R'_i **which expresses the original** Stochastic Game $\mathcal{G}$ **non-Markovian reward functions** R_i ($\forall i \in \{\epsilon, \alpha\}$)

SGRM steps in practice

1. The agent consider S' to select actions:
 - 1.1 At each time step each agent **selects the action to perform in s , simultaneously** executed.
 - 1.2 Action execution causes **game state transition** from s to s' .

$$s \xrightarrow{a_e, a_a} s'$$

2. The tuple (s_t, a_e, a_a, s_{t+1}) becomes then available.
 - 2.1 With this input, L returns the **set of satisfied atomic proposition** of $\mathcal{P}$, l_t .
 - 2.2 Accordingly to available action with l_t , a **transition** is performed over each agent RM:

$$v_e \xrightarrow{l_t} v'_e \quad v_a \xrightarrow{l_t} v'_a$$

3. RM transitions lead to each agent reward obtainance: $r_{i,t} = \sigma(v_i, L(s, a_e, a_a, s'))$

Q-functions at Nash Equilibrium

In SGRM we define the **optimal Q-function** for agent i the...

Q-function at a Nash Equilibrium

It is the **EDCR** obtained by agent i when both agents **follow a joint Nash equilibrium strategy** from the next period on. In augmented space $S \times V_e \times V_a$ it is defined:

$$q_i^*(s, v_e, v_a, a_e, a_a) = r_i + \gamma \sum_{s', v'} p(s', v' | s, v, a_e, a_a) \underbrace{\tilde{v}_i(s', v'_e, v'_a, \pi_e^*, \pi_a^*)}_{\text{Following NE from next period on}}$$

Finding optimal Policies

Given the both agent Q-function, **best response policies** are then originally extracted using the **Lemke-Howson method**:

$$\pi_e^{NE}, \pi_a^{NE} \leftarrow \text{LH_METHOD}(Q_e, Q_a)$$

...But an agent is blind to the other Q-function due to adversarial problem nature.

However, since both receive same signal (s_t, a_e, a_a, s_{t+1}) , using **personal RMs**, they estimate opponents rewards, **estimating opponent Q-Function** too:

$$\underbrace{Q_{e,e} \quad Q_{a,e}}_{\text{Agent Ego estimations}} \quad \underbrace{Q_{a,a} \quad Q_{e,a}}_{\text{Agent Adv estimations}}$$

QSG-RM Algorithm Overview

QRM-SG Algorithm Key aspects

- ϵ -greedy policy (behavioral) is adopted to balance the **exploration** and **exploitation**.

$$a_i \leftarrow \begin{cases} \operatorname{argmax}_{a \in A_i} \pi_{ii}(a \mid s, v_\epsilon, v_\alpha) & \text{with probability } 1 - \epsilon \\ \text{random} & \text{with probability } \epsilon \end{cases}$$

- Nash Q-learning for SGRM.

QRM-SG Phase 1 — Initialization

Process Breakdown (Lines 1–4)

Hyperparameter: episode length $eplength$, γ , ϵ

Input: Reward machines $\mathcal{A}_\epsilon, \mathcal{A}_\alpha$

$s \leftarrow InitialState()$; $v_\epsilon \leftarrow v_{I,\epsilon}$; $v_\alpha \leftarrow v_{I,\alpha}$

$q_{\epsilon i}(\cdot), q_{\alpha i}(\cdot) \leftarrow InitialQfunctions()$

Analysis:

- Each agent initializes **two** Q-tables to store values for self and opponent, allowing local equilibrium computation.
- SGRM (augmented) states are defined in the product space $S \times V_\epsilon \times V_\alpha$.

QRM-SG Phase 2 — Nash-Action Selection

Process Breakdown (Lines 7–13)

```
# for each episode, for each time step  $t$ :  
 $\bar{\epsilon} \leftarrow \text{GenerateRandomValue}(0, 1)$   
if  $\bar{\epsilon} < \epsilon$  then  
     $a_i \leftarrow \text{ChooseActionRandomly}()$  # Acting at random  
else  
    for  $i \in \{\epsilon, \alpha\}$  do  
         $\pi_{\epsilon i}, \pi_{\alpha i} \leftarrow \text{CalculateNashEquilibrium}(q_{\epsilon i}, q_{\alpha i})$  # Get current best strategy  
         $a_i \leftarrow \arg\max \pi_{ij}(a|s, v_{\epsilon}, v_{\alpha})$  # ... and select action accordingly  
    end for  
end if
```

Each agent solves a bimatrix stage game using the **LHmethod** to find mixed strategy equilibria.

Process Breakdown (Lines 14–18)

```
 $s' \leftarrow \text{ExecuteAction}(s, a_e, a_a) \# \text{ Agent State transition}$   
 $l_t \leftarrow L(s, a_e, a_a, s')$   
for  $i \in \{e, a\}$  do  
     $v'_i \leftarrow \delta_i(v_i, l_t); r_{i,t} \leftarrow \sigma_i(v_i, l_t) \# \text{ Game state transition and immediate rewards}$   
end for
```

- L bridges physical environmental transitions ($s \rightarrow s'$) with high level verification (l_t), allowing logic task transitions ($v_i \rightarrow v'_i$)
- Rewards $r_{i,t}$ are generated by the RM transitions, making them implicitly **history aware**.

QRM-SG Phase 4 - Nash Q-Learning

In single agent settings, Q-functions can be updated via the **off policy Q-learning**, with rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

But not if rewards depend on the joint behavior of more agents. The EDCR averaging w.r.t. policy π moves to:

$$\max_{a'} Q(s', a') \longrightarrow \mathbb{E}_{\pi_{\epsilon,i}, \pi_{a,i}}[Q_{\epsilon,i}(s', a')]$$

Nash-Q-Learning makes use of co-dependent expectation. Each agent i updates its estimates:

$$Q_{\epsilon,i}(s, \mathbf{a}) \leftarrow Q_{\epsilon,i}(s, \mathbf{a}) + \alpha(r_{\epsilon,i} + \gamma \mathbb{E}_{\pi_{\epsilon,i}, \pi_{a,i}}[Q_{\epsilon,i}(s', \mathbf{a}')] - Q_{\epsilon,i}(s, \mathbf{a}))$$

$$Q_{a,i}(s, \mathbf{a}) \leftarrow Q_{a,i}(s, \mathbf{a}) + \alpha(r_{a,i} + \gamma \mathbb{E}_{\pi_{\epsilon,i}, \pi_{a,i}}[Q_{a,i}(s', \mathbf{a}')] - Q_{a,i}(s, \mathbf{a}))$$

QRM-SG Phase 4 - Nash Q-Learning (cont'd)

Current policies estimations are obtained with a call to **CalculateNashEquilibrium** function.

Compared to classical Q-Learning, the algorithm still looks ahead to the **next state** (s', v'_e, v'_a), but instead of looking at the entry with maximal value wrt action, it **averages with respect to all combinations of agents actions**, weighted by the **joint probability** of action choice.

Additional Keypoints

Introduction of the counterfactual Reasoning

Original paper Nash Q-learning is as a function of the augmented state space $S \times V_{\epsilon} \times V_{\alpha}$. Additionally, given a state, step update affects only the state $(s, v_{\epsilon}, v_{\alpha})$ after action a_{ϵ}, a_{α} .

Pointed out problem

Given a physical state s , we can reach next s' having **many different** RM internal states v_{ϵ}, v_{α} .

Since, the 4 Q-function updates highly depends on $|V_{\epsilon} \times V_{\alpha}|$, to perform a **general state game learning**, we would need to **experience a huge amount of steps/episodes**.

Introduction of the counterfactual Reasoning

However, physical states (s) and logical game states (v_e, v_a) are **independent**:

- Agents may move in gridworld and receive I_t **regardless of task related history**.
- RM states are meant to keep **only task relevant high-level informations** through v_i , **without spatial** nor **temporal** informations.

So, during each $s \rightarrow s'$ transition, we can **simulate** having **different RM states**, i.e. simulating **alternative historical backgrounds** at each $s \rightarrow s'$.

Counterfactual Reasoning

Instead of needing performing $s \rightarrow s'$ with every possible **history** permutation, we use experience all histories at once.

This shifts (*at its best*) complexity **away from exploration** and transfers it into the **computation of an individual trace**.

Counterfactual Reasoning (cont'd)

Counterfactual Reasoning

Since the **practical environment transition is independent from the RM state**, instead of updating the Q-value for state (s, v_e, v_a) **only**, we can

- Consider all RMs states $\tilde{v}_e, \tilde{v}_a$, simulate being in augmented state $(s, \tilde{v}_e, \tilde{v}_a)$.
- Simulate state and the RM transition to $\tilde{v}'_e, \tilde{v}'_a$ if s' is reached, i.e. reaching $(s', \tilde{v}'_e, \tilde{v}'_a)$
- Update the Q-function associated to $(s, \tilde{v}_e, \tilde{v}_a)$

Original Assumptions

Assumption 1: Exploration

Every state $s \in S$, RM states $v_e \in V_e$, $v_a \in V_a$, and actions $a_e \in A_e$, $a_a \in A_a$, are visited infinitely often when the number of episodes goes to infinity.

Assumption 2: Learning Rate

The learning rate α_t satisfies the following conditions for all t :

1. $0 < \alpha_t < 1$, $\sum_t \alpha_t(s, v_e, v_a, a_e, a_a) = \infty$, $\sum_t [\alpha_t(s, v_e, v_a, a_e, a_a)]^2 < \infty$, and the latter two hold uniformly and with probability 1.
2. $\alpha_t(s, v_e, v_a, a_e, a_a) = 0$ if $(s, v_e, v_a, a_e, a_a) \neq (s^t, v_e^t, v_a^t, a_e^t, a_a^t)$.

Assumption 3: Stage Game Structure

One of the following conditions holds during learning:

1. Every stage game $(q_{ei}(s, v_e, v_a), q_{ai}(s, v_e, v_a))$, $i \in \{e, a\}$, for all t, s, v_e, v_a , has a **global optimal point** $(\tilde{\pi}_{ei}, \tilde{\pi}_{ai})$ (i.e., $\tilde{\pi}_{ji}q_{ji} \geq \tilde{\pi}'_{ji}q_{ji}$ for any $\tilde{\pi}'_{ji}$, $j \in \{e, a\}$) and agents' rewards in this equilibrium are used to update their Q-functions.
2. Every stage game $(q_{ei}(s, v_e, v_a), q_{ai}(s, v_e, v_a))$, $i \in \{e, a\}$, for all t, s, v_e, v_a , has a **saddle point** $(\tilde{\pi}_{ei}, \tilde{\pi}_{ai})$ (i.e., $\tilde{\pi}_{ji}\tilde{\pi}_{-ji}q_{ji} \geq \tilde{\pi}'_{ji}\tilde{\pi}_{-ji}q_{ji}$ for any $\tilde{\pi}'_{ji}$, $\tilde{\pi}_{ji}\tilde{\pi}_{-ji}q_{ji} \leq \tilde{\pi}_{ji}\tilde{\pi}'_{-ji}q_{ji}$ for any $\tilde{\pi}'_{-ji}$, $j \in \{e, a\}$), and agents' rewards in this equilibrium are used to update their Q-functions.

Game Results and Experiments

Testing Environment: Evaluated on 6×6 competitive, sparse-reward grid world variations of Pac-Man.

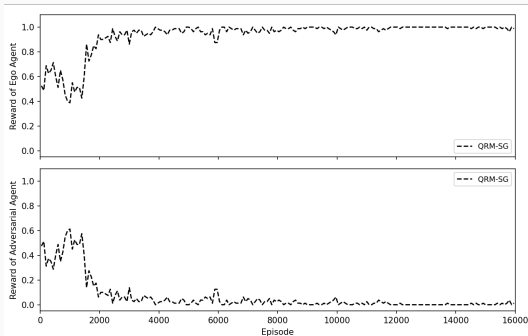
Comparative Baselines:

- Nash-Q: Pure spatial state-action tracking.

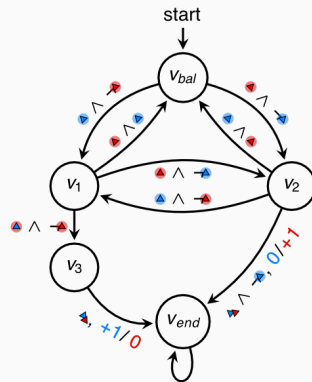
Case Study Empirical Analysis

- **Case Study I :** In the first task we require the agents to firstly power themselves up, then destroy the other agent base and finally eat the other agent.
- **Case Study II :** This task has the same requirements that the first task but the agents must power themselves up again after destroying the other agent base and before eating the other agent.
- **Case Study III:** Same as task II but the starting position of adv agent is randomized between two positions.

Case Study I



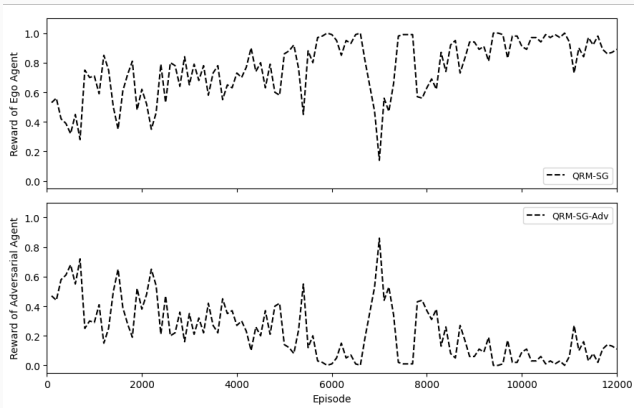
(a) Task I random initialization: Reward Trajectory



(b) Reward Machine for Task I

The plot shows the average reward trajectory every 100 episodes, we can see that the QRM-SG algorithm converges to the optimal policy at around episode 7500.

Case Study I: Nash Q-learning

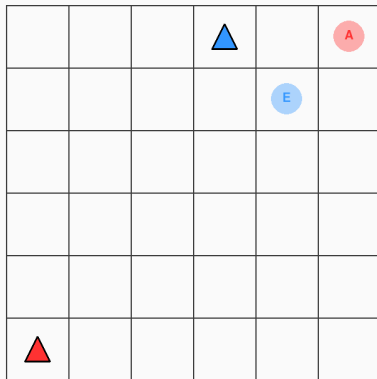


(a) Task I: Reward Trajectory

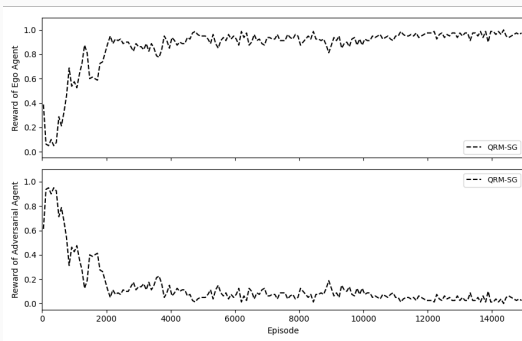
Analysis:

- High Variance: Sharp, frequent drops suggest policy instability rather than smooth learning.

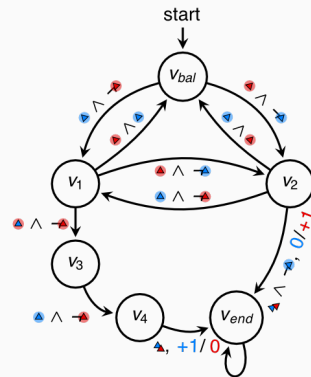
Task I



Case Study II



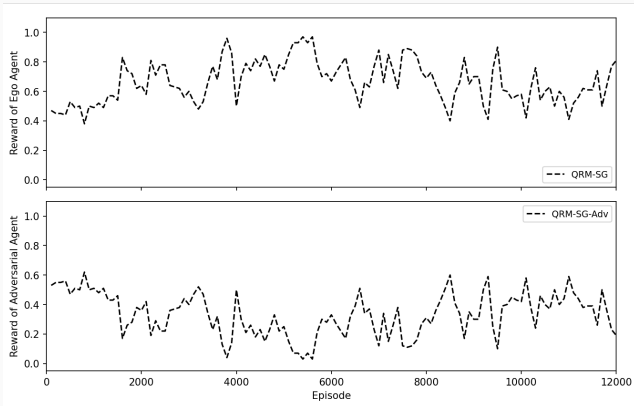
(a) Task II: Reward Trajectory



(b) Reward Machine for Task II and III

We can see that the QRM-SG algorithm converges with some more difficulties around episode 10000.

Case Study II: Nash Q-learning

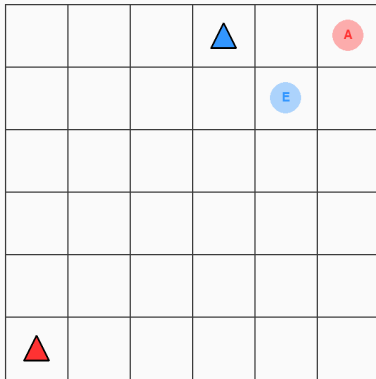


(a) Task II: Reward Trajectory

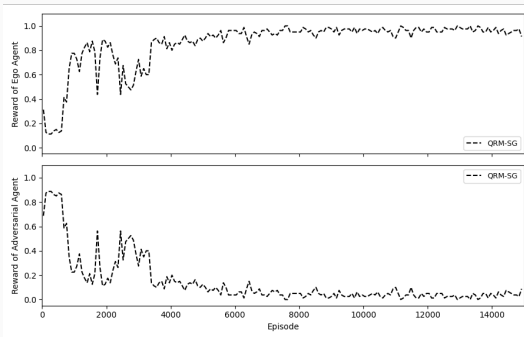
Analysis:

- We can see the same problems found in the first task, with a high variance and a very unstable learning process.

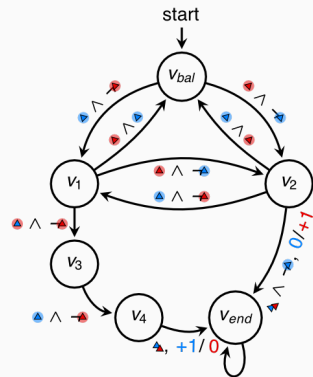
Task II



Case Study III



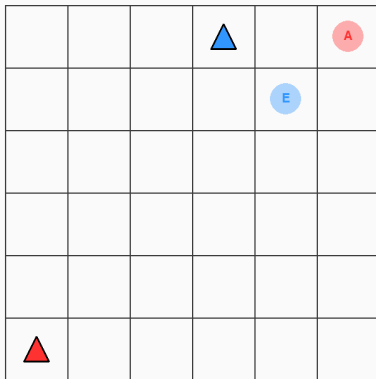
(a) Task III: Reward Trajectory



(b) Reward Machine for Task II and III

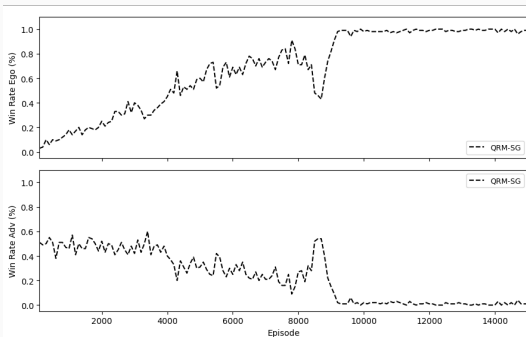
We can see that the QRM-SG algorithm converges around episode 8000.

Task III

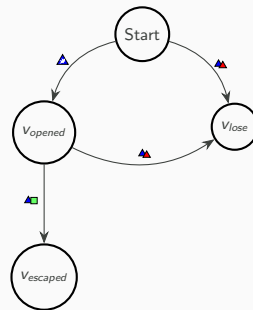


Maze experiment

Maze I



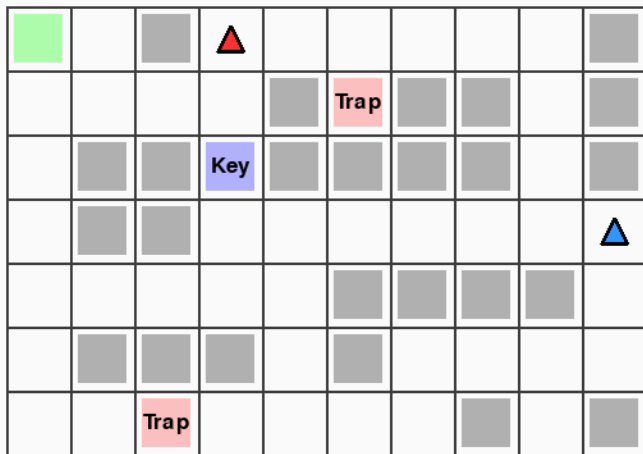
(a) Maze I: Reward Trajectory



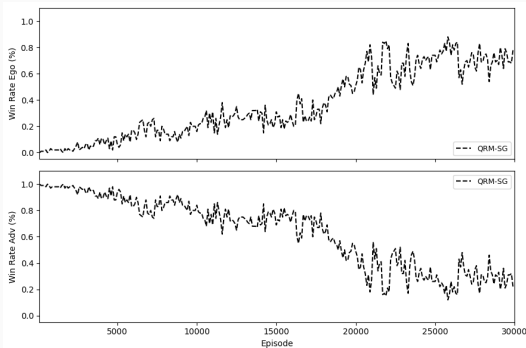
(b) Reward Machine

We test QRM-SG algorithm on another task, the agent must find the key, hold it and use it to escape while avoiding agent Adv. As we can see we have convergence to the optimal policy around episode 9800.

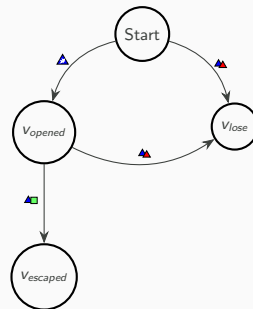
Simulation: Maze Execution



Maze II



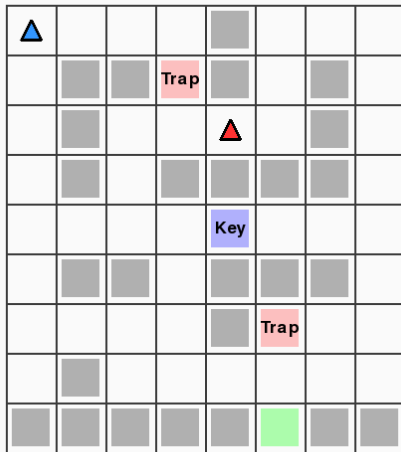
(a) Maze II: Reward Trajectory



(b) Reward Machine

The algorithm has some difficulties in finding the optimal policy, however the ego agent is actually learning how to avoid agent Adv, collect the key and finally escape. More episodes would have likely led to a better convergence.

Simulation: Maze Execution



Conclusion

Summary & Future Outlook

- **Core Contribution:** Bridges reward machines with MARL to model non-Markovian constraints in stochastic games.
- **Robust Architecture:** Augmented state transformations allow tabular actors to navigate complex environments successfully.

References

- [1] Jueming Hu, Jean-Raphaël Gaglione, Yanze Wang, Zhe Xu, Ufuk Topcu, and Yongming Liu.

Reinforcement learning with reward machines in stochastic games.

arXiv preprint arXiv:2305.17372v3, cs.MA, 2023.